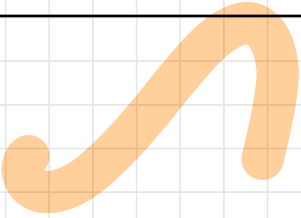


Why should we use Views? Why not directly through the tables?

- Views provide an interface for specific users. We can allow a user to only see the information they need. Allow for Privacy
- View are slightly slower than direct table access because it needs to retrieve previous commands and translate the view code to the table.

So, to decide what to use is priority of the task. Do you want a privacy or a faster a response time?

- However, in the modern world, views are only slow on the first few runs because of optimization techniques that allow them to be more efficient.
- Queries use interpreters instead of compilers because there might have errors internally. Eg: missing keys / values



Mid-Term Q1

✓ 1. Given the Supplier-Part database with 3 separate table structures: S (S#, SNAME, CITY), P (P#, PNAME, COLOR), and SP (S#, P#, QTY).

Explain why should we use the 3 separate tables instead of combining them into only one big table SP' (S#, SNAME, CITY, P#, PNAME, COLOR, QTY)? (10 Marks)

S(S#, SNAME, CITY), P(P#, PNAME, COLOR), SP(S# P# QTY)
We should split the 3 tables in this manner to avoid Insert and Delete problems.

For example;

S#	SNAME	CITY	P#	PNAME	COLOR	QTY
S1	John	YGN	P1	Bolt	White	20
S1	Mary	MDY	P2	Nut	Red	30
S2	David	MGY	P3	Bolt	Red	15

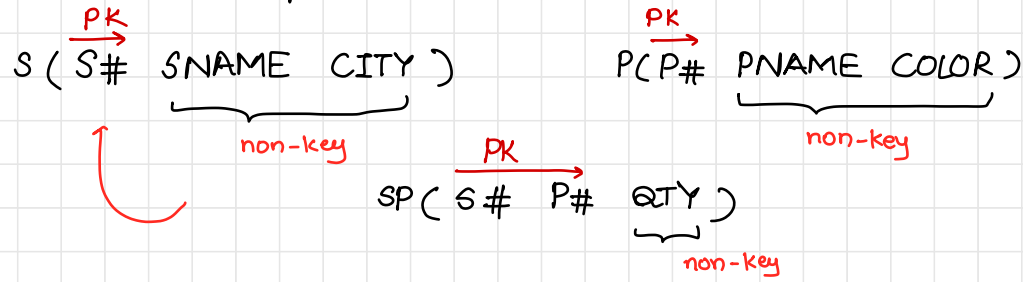
If we want to insert a supplier that does not have any parts;
S3 Mary BKK - - - - (- is NULL); it leads to Redundancy.

If we want to remove (S1 John) row from the relation, the information of (P1 Bolt White 20) will be lost. Facts are unintentionally removed by the operation.

To Solve such insert & delete problems, Normalization is required.

The large example given by the question (SP'), correctly represents a relation in tabular format. Therefore, it is in the 1NF.

Table SP' in the 1NF still has insert and delete problems. So, we must split and compare with 2NF definition.



2NF: every non-key is fully dependent on the primary key.

Therefore, table S, P & SP are in the 2NF. Insert & Delete problems are solved.

② What is Relation & two representation of Relations?

A relation is a subset of Cartesian Product of Domains.

Domain : Name = { John, Mary }
Age = { 16, 19 }

name x age = { (John, 16), (John, 19), (Mary, 16),
(Mary, 19) }

subset of Cartesian Product; NA = { (John, 19), (Mary, 16) }
1st rep

2nd representation;

NA	Name	Age
	John	19
	Mary	16

③ What are properties of tables which repre a relation ?

① An attribute must belong to only one domain.

(One Col for one topic only)

② Column values must be atomic. { 09-422, 06-253, 09-779 }

(Each col value must be a single value, not collection.)

③ No 2 rows are the same, D = { D1, ~~D1~~, D2, D3 }

Each row must be unique.

④ Rows have no order.

{ D1, D2, D3 } = { D3, D2, D1 }

④ Show two Tables which are relations!

Student

S#	SName	City	Status
S1	John	A	100
S2	Mar	B	200
S3	Susu	C	300
S4	Koko	D	400

SP

P#	S#	QTY
P1	S1	2
P2	S2	3
P3	S3	1
P4	S4	5

⑤ Tables which are not Relation!

S'

S#	COLNAME	VALUE
S1	SNAME	John
S1	CITY	London
S1	STATUS	100

C

C#	CNAME	Tel
C1	John	{12,34,35}
C2	May	{257,253}
C3	Tee	{123,456}

value column contains multiple domains.

Tel columns value are not atomic.

⑥ Super Key

A super key is an attribute or set of attributes that uniquely identify a tuple (row).

Emp

EID	NAME	Email
1	J	j@gmail.com
2	A	a@gmail.com
3	B	b@gmail.com
4	C	c@gmail.com

SuperKeys:

EID & Email ✓
Combination of ✓

EID, Name, Email

⑦ **Primary key** is a selected candidate key.

A **candidate key** is a super key which does not have other super keys as its subset.

From Emp Table, EID & Email can be candidate keys.

EID can be primary key but not Email because it might change over time.

⑧ **Foreign key**

→ an attribute that is not primary key.

A foreign key is a **non-key attribute** of a Relation which is a primary key of other relations.

S	Primary	non-key
	S#	SName P#
	S ₁	John P ₁
	S ₂	May P ₂

P	Primary
	P# PName QTY
	P ₁ Su 2
	P ₂ Bo 5

Alternate key is a candidate key, that is not a primary key.

Composite Or Combined key is a key that comprises more than one attribute.

$S = [S\# \text{ SNAME CITY STATUS}]$

S# is Candidate key. SNAME is also Candidate key.

→ S# chosen to be **Primary key**. Then; SNAME is **Alternate key**.

$SP = [S\# P\# QTY]$

$(S\#, P\#)$ is a candidate key that comprises two attributes, **Composite key**.

Null : ① Attribute Applicable but value unknown

② Attribute Not Applicable

Surrogate Key :

A system-generated or application-generated unique identifier.

⑨ **Relational Complete Language (SQL)**

- is a language which is at least as powerful as relational algebra or relational calculus.

⑩ **Relational DataBase Model** : Logical Represent of DataBase for Apps and programs to work with

Data Structure : Domain (Atomic Values) ; n-ary relations

Data Integrity : Pri Key must not be null
Foreign Key must match primary key (be null)

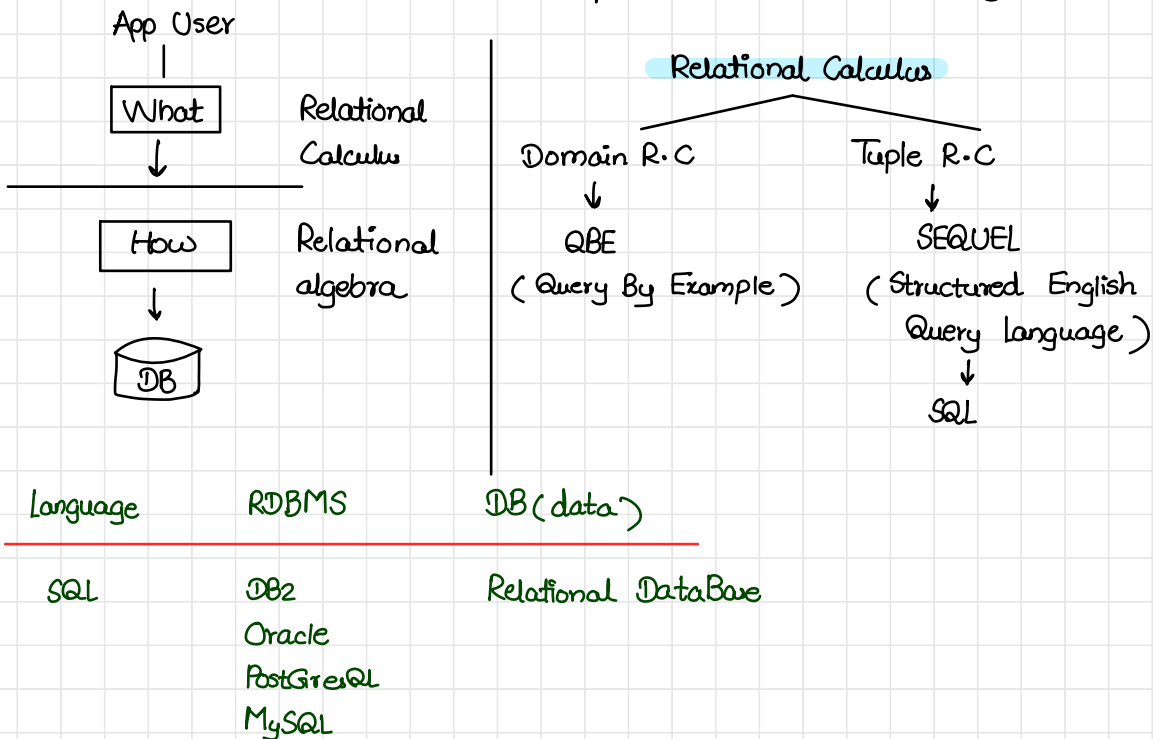
Data Manipulation : Relational Calculus (Intersection, Difference, Union, Divide) | Languages that work on logical DataBase Structure

Database Models

→ In database context, what is a relation?
- Data rows excluding the table structure

- ① **Data Structure** : Logical Data Structure which represents data & RS
- ② **Integrity Rules** : Rules which enforce correctness on data structure
(not Business Integrity Rules)
 - **Entity Integrity** → primary key | alternative (not selected as P.K.)
super key
candidate key
simple key has only one attribute.
combined key or composite key more than one att
 - **Referential Integrity** : Foreign key

- ③ **Data Manipulation** : Languages that work on the logical Data Structure
 - **Database lang** : Relational Complete language
- at least as powerful as relational algebra or cal



② Good Characteristics of Database System

1. High Productivity Computing / Data Model, Language & Tools
2. High Degree of Data Independence

↙
- means that when changes are made to the (logical & physical database structure) should not lead to changes of applications programs.

└─ Phy changed → logical unchanged
 Phy changed → App Pro unchanged

2.1; Physical Data Independence

data structure
- Changes made to physical database does not affect the conceptual or logical database.

2.2; Logical Data Independence

└─ logical changes — Exter Unchan
 Logical Changes — Phy Pro Unchange
- Changes made to logical database does not affect the external database.

3. High degree of Data Security / Privacy :

Using views adds privacy and the DBMS manages the access to itself.

- In principle, the DBMS should handle the User IDs & passwords, with no subversion
- Subversion : no direct access from user to Table, only through views

4. High Degree of Data Integrity

- Correctness of Data
- Information on the database must be correct according to the integrity rules (constraints, validation rules)
- Best practice : Rules should be declared on the database & be enforced by the DBMS

⑤ Data Management Facilities & Tools - Tools provided by the DBMS

- Concurrency Control - Controls the multi-user access
- Recovery (Recover the data)
- Query Optimization
- Data Distribution

⑥ Logically Centralized Control : Database system should be managed by a central team called the Database Administrator Team (DBA).

- Centralized Vs Distributed , centralized data models are now the norm
 - The DBA Team manages the rules and data in the database system.
 - The physical control can either be centralized or distributed. Centralized is better.
-

- ⑬ Query Optimizer
- ↓
- ① Rule-Based Optimizer
 - ② Cost-Based Optimizer →
 - ③ Semantic Query Optimization

- is the module of the DBMS which determines the best physical route to the result.

⑭ Rule-Based Optimizer

- employs a fixed set of rules which often refer to the way database statements (SQL statements) are written.

⑮ Cost-Based Query Optimizer

- the best route by using database statistics.

DB statistics includes size of index, no. of rows returned, size of table etc.

⑯ Semantic Query Optimizer (SQO)

- employs integrity rules (business rules) for Query Optimization.

Redundancy : a fact is stand out than others (in the database)

Problems Of having Redundancies in a database table

- ① Waste of Storage Space (not so much nowadays)
- ② Update Anomaly
 - a. can be solve by current technologies
 - b. `UPDATE S Set CITY = " LONDON " where S# = ' S1 "`
- ③ Insertion Anomaly
 - a. Some facts can't be inserted because they are against the integrity rule.
 - b. Some inconsistent facts can be inserted - Data Inconsist
- ④ Deletion Anomaly

The removal of a fact via delete operation may lead to the removal of other facts unintentionally.